

A Partially Reconfigurable Preprocessing Layer to Mitigate CNN Complexity

Rasha Karakchi and Noah Robertson

University of South Carolina, Columbia SC 29208, USA

karakchi@cec.sc.edu, noahhr@email.sc.edu

Abstract. Deep learning Convolutional Neural Network (CNN) is a powerful tool for feature extraction, edge detection and image classification. However, the training system of the networks has become increasingly complex due to the continuous growth in dataset which significantly impacts the computational time and energy consumption. In domain-specific platforms, this system is bounded by another level of limitation which is the hardware resources and the I/O resources. Using Gabor filter as a preprocessing layer can eliminate the redundant features and unnecessary information which results in much fewer memory used and less computation involved. In this paper, we propose a preprocessing layer which consists of Gabor kernel that serves as a weight kernel for CNNs and convolutional kernel that produces the features. We implemented the two kernels using the hardware description language Verilog that targets Xilinx FPGA devices. We designed a novel Gabor kernel based on the reconfigurability of FPGA by utilizing the BlockRAM (BRAMs) to store Gabor parameters, SRAM-based memory to store the input image and Digital Signal Processing (DSPs) for convolutional computation. We evaluated the design on Virtex 5 FPGA device using a 64x64 image, and we measured the performance of the Gabor kernel and convolutional kernel individually and found that Gabor kernel spend $\sim 0.1\mu s$ to produce the weight kernel and convolutional unit spends $\sim 0.8\mu s$ to find features.

Keywords: Spiking Neural Network, Leaky Integrate-and-Fire, scalability, FPGA, accelerators.

I. Introduction

Deep learning Convolutional Neural Networks (CNNs) have widely been used to solve problems related to machine learning and computer vision [1]. However, the large-scale structure and the associated training complexity present CNNs as one of the most compute-intensive workloads across all modern computing platforms [2]. This results in slowing down the computation process and lowering the overall performance. Domain-specific platforms such as Field-Programmable Gate Arrays (FPGAs) are well-known for their suitability to execute CNNs efficiently because of their high level of parallelism and the large number of Digital Signal Processing (DSPs) available on the chip [3]. However, the exponential growth in data size has made many of the existing designs and methodologies ineffective and hardware resources not enough for CNN's operation. Therefore, reconfigurable computing technology has leveraged in order to utilize all types of memory built-in on such devices and exploit the capability of reconfigurability (reuse components) [4]. BlockRAM is an on-chip fast memory that is used to store large amounts of data inside the FPGA [10]. SRAM is a volatile memory which holds their configuration patterns in static RAM memory cells [10]. On the other hand, the data growth has resulted in an increase in data redundancy. For this reason, using filters in CNN systems has become essential to remove unnecessary data, and in turn, mitigating CNN complexity [5]. Gabor filter is known for its use in computer vision. Gabor filter is a linear filter which executes two exponential functions as illustrated in Eq.1. The first function represents the origin-centered Gaussian function multiplied by a sinusoidal wave [6]. The characteristics of Gabor filter are controlled by three main parameters: frequency, bandwidth, and orientation of x' and y' as shown in Eq.1. While x and y are the coordinates of the input pixel, λ is the wavelength of sinusoidal function, θ represents the angle of the orientation of the normal to the parallel stripes of a Gabor function, Ψ is the phase offset, σ is the standard deviation of the Gaussian envelop and γ is the spatial aspect of the real part.

$$g(x, y, \lambda, \theta, \Psi, \sigma, \gamma) = \exp\left(-\frac{x'^2 + \gamma^2 y'^2}{2\sigma^2}\right) \exp\left(2\pi \frac{x'}{\lambda} + \Psi\right) \quad (1)$$

Several research works focused on designing customized Gabor filter on FPGA for specific applications [3-9]. In this paper, our goal is mitigating CNN complexity through designing a preprocessing layer representing a parameterizable partially reconfigurable Gabor kernel. The main task of this layer is to remove any redundant values in a fast reusable way and create a weight kernel that convolutes with the input image to output the features. Our designed system consists of the preprocessing layer (Gabor kernel) and the processing (computing)

layer (convolutional kernel). We designed Gabor kernel as a novel partially reconfigurable FPGA design where the BRAM is used to reconfigure the parameters, SRAM is used to store the input image and DSP components are used for floating-point computations. The convolutional kernel is designed using DSP components. Our proposed approach is discussed in Section II and the evaluation and conclusion are discussed in Section III.

II. Proposed Approach

Our research contribution involves a novel design for Gabor kernel and the convolutional kernel using Verilog language which target Virtex 5 FPGA device. Below is the description of each kernel.

A. Gabor Kernel: We designed Gabor kernel as a parameterizable accelerator that represents the weight kernel (window) which will be created based the given parameters. As shown in Fig.1, the accelerator itself consists of three main units that run consecutively: *pi_sinx_siny* (orange), *Gaussian* (light blue) and *sinusoidal* units (green). The Gaussian function is represented by the real part $x' = x \cos \theta + y \sin \theta$ and the imaginary part $y' = -x \sin \theta + y \cos \theta$. Each unit executes a part of Gabor kernel function (Eq.1) and implemented using DSP components on FPGA. We used Taylor series to execute Cosine, Sine, Exp functions. *ParRAM* component (dark blue) is Block RAM that is used to store the parameters' values where each parameter is located in a specific address of the memory as shown in Table 1. The input buses to *ParRAM* represents the addresses (add) of each parameter in the BRAM.

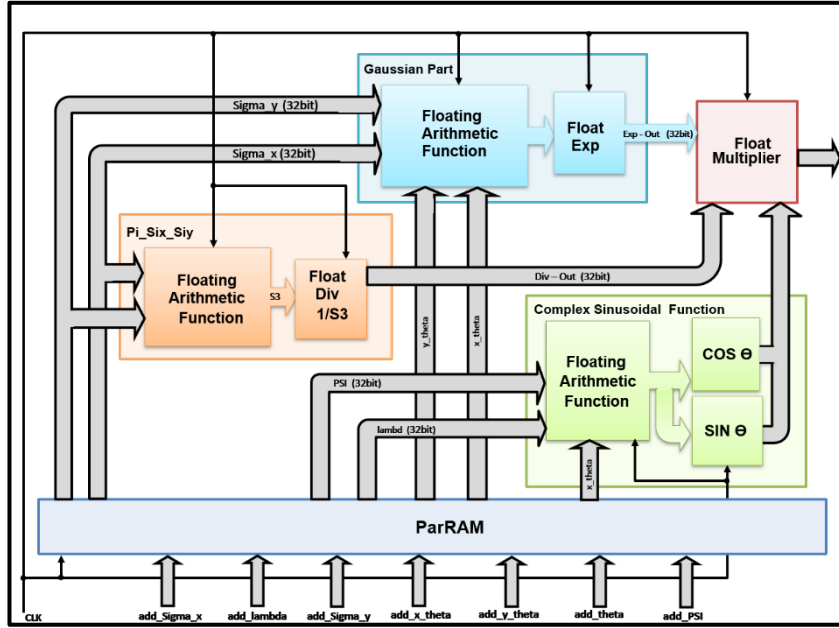


Fig. 1. Implementation of Gabor-Kernel Architecture

- B. Convolutional Kernel:** Our convolutional unit performs the 2D convolution operation [11] based on Eq.2. As shown in Fig.2, the unit reads the small matrix of weight generated by the Gabor-Kernel unit. The kernel slides over the 2D input image matrix performing an elementwise multiplication with the part of input it is currently on, then summing up the results into a single output pixel as shown in Fig.2a. The kernel repeats this process for every location it slides over, converting a 2D matrix of features into yet another 2D matrix of features (shown in Fig.2d).

$$w(x, y) * f(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x - s, y - t) \quad (2)$$

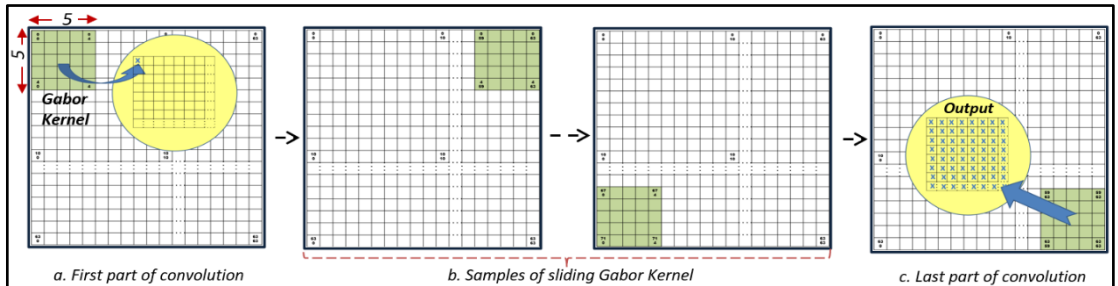


Fig. 2. The operation of 2D convolution unit on a 64x64 input image and 5x5 Gabor weight kernel

III. Evaluation and Discussion

Fig.3 shows our design flow starting by the 64x64 image stored in SRAM memory. We set up Gabor parameters as shown in Table 1. We used ISE Design Suite to measure the required time to run Gabor kernel and convolutional kernel separately. Our Gabor kernel design spent about 100ns to generate the weight kernels and about 800ns for the convolutional kernel to generate the features. This is a preliminary work that attempts to answer an important research question which is the CNN complexity growth in respect to data growth. The results we obtained are promising, therefore for future work we plan to explore the following. First, we plan to test the design on actual benchmarks and compare it with state-of-the-art methods. We also plan to expand the design to support multi-scale and multi-directional Gabor filters, which will improve the overall performance. Furthermore, we plan to run the design on Ultra scale FPGA devices to perform larger network and bigger image.

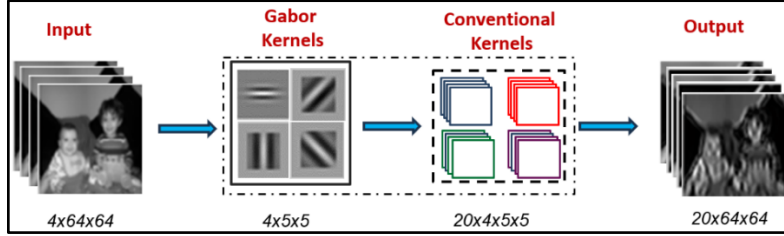


Fig. 3. An example of feature extraction for a 64x64 image using our proposed preprocessing layer

Table 1. ParRAM addresses and values.

Assigned parameters	Assigned addresses	Values used for testing
α_x	0	0.1
α_y	1	1
θ	2	0,45
λ	3	5,7
φ	4	0
γ	5	0.5
x_theta	6-30	
y_theta	31-55	

References

1. Chai, Junyi, et al. "Deep learning in computer vision: A critical review of emerging techniques and application scenarios." *Machine Learning with Applications* 6 (2021): 100134.
2. Glorot, Xavier, and Yoshua Bengio. "Understanding the difficulty of training deep feedforward neural networks." *Proceedings of the thirteenth international conference on artificial intelligence and statistics. JMLR Workshop and Conference Proceedings*, 2010.
3. Basalama, Suhail, et al. "FlexCNN: An End-to-End Framework for Composing CNN Accelerators on FPGA." *ACM Transactions on Reconfigurable Technology and Systems* 16.2 (2023): 1-32.
4. Kim, Heekyung, and Kyuwon Ken Choi. "A Reconfigurable CNN-based Accelerator Design for Fast and Energy-Efficient Object Detection System on Mobile FPGA." *IEEE Access* (2023).
5. Alekseev, Andrey, and Anatoly Bobe. "GaborNet: Gabor filters with learnable parameters in deep convolutional neural network." *2019 International Conference on Engineering and Telecommunication (EnT)*. IEEE, 2019.
6. Henriksen, Jesper Juul. "3D surface tracking and approximation using Gabor filters." *South Denmark University* 28 (2007).
7. Aparna, Yarramaneni Leela, et al. "FPGA Based Plant Disease Identification using Convolution Neural Networks." *2023 IEEE 8th International Conference for Convergence in Technology (I2CT)*. IEEE, 2023.
8. Li, Wenmao. "A video vibration measurement core based on FPGA." *Third International Conference on Computer Vision and Pattern Analysis (ICCPA 2023)*. Vol. 12754. SPIE, 2023.
9. Y. -W. Wang, G. G. C. Lee, Y. -H. Chen, S. -Y. Chen and T. -P. Wang, "Implementation of Gabor Filter Based Convolution for Deep Learning on FPGA," *2022 IEEE International Conference on Recent Advances in Systems Science and Engineering (RASSE)*, Tainan, Taiwan, 2022, pp. 1-4, doi: 10.1109/RASSE54974.2022.9989881.
10. <https://www.xilinx.com/products/silicon-devices/fpga/virtex-7.html>
11. <https://towardsdatascience.com/intuitively-understanding-convolutions-for-deep-learning-1f6f42faee1>